



SCHOOL OF INFORMATION TECHNOLOGY

Module code	WEBA262
Module name	Web Development 2A
Assessment code	SS2
Assessment type	Individual Assignment
Due date	17 April 2026
Total marks	126 x 0.953 = 120

- This assignment document is subject to change. Always refer to the latest version available on the Learning Management System (LMS) - Canvas or ECI.
- Consult the Learning Management System (LMS), i.e., Canvas or ECI, for the due dates, the Assessment Guidelines for Students document, the Assignment Overview, and other updated detail. Check the Announcements by your lecturer and use the Module Q&A forum to ask questions related to this assignment.
- For rules and regulations applicable to assessments, refer to the Student Handbook.
- Note STADIO's rules relating to plagiarism and cheating. Do not hesitate to seek help if you are unsure of how to reference your sources. Make use of the STADIO Referencing Guide, available from the Online Library.

INTRODUCTION

The goal of this module is to provide students with knowledge of JavaScript programming and interacting with controls on the client side. In this project assessment, you will create a web application using various JavaScript functions.

Specifically, you will develop an interactive game that tests a user's memory or reaction and mouse skills. Each student will be assigned a unique game scenario.

OVERVIEW

The complete solutions will be provided for five mini games. Study these solutions to understand what is expected for the larger game that you will develop.

You will be provided with the html and css files for the game that is assigned to you. You are welcome to adapt these files if needed but note that no marks will be assigned for that. The focus of this module is on JavaScript.

Eventually your project will contain these files:

- index.html (launcher / settings page)
- instructions.html (Specific instructions per game)
- game.html (game interface)
- styles.css (applicable to both html files)
- launcher.js (Goes with index.html)
- game.js (Goes with game.html)

Task	Title/ Description	Marks
1	Variables, Objects, Arrays	28
2	Functions	19
3	Forms	15
4	Storage	20
5	Feedback	14
6	Overall working	30
	Total	126

ARTIFICIAL INTELLIGENCE TRAFFIC LIGHT MATRIX (AITLM)

	RED	AMBER	GREEN	
Task(s)	May NOT use AI	May use AI	MUST use AI	Detail of AI use
1 – 5				You may use AI to explain concepts, but you may not use AI to generate code, and neither may you copy and paste code fragments directly from any source. Keep record of all prompts to AI and its responses as you might be asked afterwards to provide evidence of such actions.
NOTE: • STADIO employs AI writing detection tools to monitor AI use. • Presenting work produced by AI as your own contravenes the STADIO Plagiarism policy. • Refer to the STADIO Referencing Guide for guidance.				

INSTRUCTIONS

INSTRUCTIONS: ALL GAMES

Deliverables

Your code must be contained in six files:

- index.html (launcher / settings page)
- instructions.html (Specific instructions per game)
- styles.css (applicable to both html files)
- launcher.js (Goes with index.html)
- game.html (game interface)
- game.js (Goes with game.html)

Form inputs

Your launcher page must include:

- Text input (e.g. player name)
- Dropdown (difficulty or game options)
- Radio buttons (theme or mode)
- Checkboxes (optional features)
- Buttons (start, reset, save, load)

JavaScript requirements

You must demonstrate:

- Event handling (click, change, input, submit)
- Functions with parameters
- JavaScript objects (player, game state, etc.)
- Use arrays to manage game elements, for example
 - game elements (e.g. objects, cards, positions)
 - score history or game events
 - dynamic game state
- Use of built-in functions:
 - Math (e.g. randomness, scoring)
 - String (e.g. formatting, validation)
- Text input (e.g. player name)

Data storage

- Cookies. Store at least:
 - Player name
 - Best score
- Session storage. Store:
 - Current game state (score, round, etc.)

Dialog boxes

- alert(). Feedback, e.g. game over
- confirm(). For example confirm reset of game.
- prompt(). For example to get player's name if not entered.

UI / UX requirements

- Clear layout
- Responsive design
- Visible game status (score, status, etc.)
- User-friendly interactions

Game window

- The game must open in a separate window (game.html)
- Must include:
 - Start button
 - Reset button
 - Back to settings button

General

- Ensure that the system provides clear prompts and outputs to guide the user.
- Provide ample comments to explain your logic
- Use meaningful variable and function names
- Apply proper indentation and formatting
- Avoid code duplication. Use functions.
- Do not include unused or commented-out code
- The application must run correctly on a browser.
- Runtime errors are taboo!!!

INSTRUCTIONS: PER GAME

Every student will get a different game. Follow the instructions for the game that is assigned to you. The provided instructions.html file for every game contains more details on each game.

1. Sky Catcher

- Move a bat left/right
- Catch falling objects before they hit ground
- Difficulty → falling speed
- Bonus items give extra points

2. Memory Match Arena

- Flip cards to find matching pairs
- Difficulty → grid size
- Track moves and time

3. Whack-a-Mole Arena

- Click moles as they appear
- Difficulty → speed and visibility time
- Use arrays for holes, mole history and mole types
- Bonus mole gives extra points

4. Balloon Pop Challenge

- Balloons rise upward
- Player clicks them before they escape
- Difficulty → speed and spawn rate
- Missed balloons reduce score

5. Target Click Game

- Targets appear randomly on screen
- Disappear after short time
- Difficulty → size + time visible
- Clicking empty space counts as a miss

6. Simon Says Sequence

- System shows a sequence
- Player repeats it
- Sequence grows each round
- Difficulty → speed and sequence length

7. Number Pair Match

- Match related pairs (e.g. 4 and “four”, or 2+3 and 5)
- Difficulty → more pairs
- Must define matching logic clearly

8. Falling Fruit Sorter

- Objects fall from top
- Player catches only correct items
- Difficulty → speed + mixed object types
- Wrong catch reduces score

9. Maze Escape Grid

- Move player through grid maze
- Avoid traps and reach exit
- Difficulty → maze size and obstacles

10. Reaction Time Tester

- Signal appears randomly
- Player must react quickly
- Measure reaction time
- Multiple rounds, average score

11. Dodge the Obstacles

- Player avoids moving hazards
- Survival time = score
- Difficulty → number and speed of obstacles

12. Pattern Copy Game

- Show pattern briefly
- Player recreates pattern
- Difficulty → grid size + display time

13. Treasure Box Guessing Game

- Grid of hidden boxes
- Some contain rewards, others penalties
- Difficulty → number of traps
- Limited attempts

INSTRUCTIONS: SUBMISSION

Zip the entire project folder in a single zip file. Name the file as WEBA262_SS2_<student number>_<surname>.zip. Replace <student number> and <surname> with your own student number and surname, for example WEBA262_SS2_123456_Johnson.zip.

GUIDANCE

Refer to the Rubric below, to understand the assignment performance expectations and how your work will be assessed. Once you have completed your assessment, use the checklist below to ensure that you have covered all the required elements.

RUBRIC

Each one of the listed aspects below will be assessed as follows:

	Not present	Lacking	Good	Excellent
Variables				6
- Declared	0	1	2	3
- Data retrieved	0	1	2	3
Objects				12
- Present	0	1	2	3
- Constructor	0	1	2	3
- Properties	0	1	2	3
- Methods	0	1	2	3
Arrays				10
- Created	0	1-2	3-4	5
- Data retrieved	0	1-2	3-4	5

Functions				19
- Basic functions	0	1	2	3
- Functions with return values	0	1	2	3
- Functions with parameters	0	1	2	3
- Built-in String functions	0	1-2	3-4	5
- Built-in Math functions	0	1-2	3-4	5
Forms				15
- Events (onSubmit, onChange, onClick, etc.)	0	1-4	5-8	9-10
- Validation	0	1-2	3-4	5
Cookies				10
- Setting values	0	1-2	3-4	5
- Retrieving values	0	1-2	3-4	5
Session storage				10
- Setting values	0	1-2	3-4	5
- Retrieving values	0	1-2	3-4	5
Feedback				14
- alert	0	1	2	3
- confirm	0	1	2	3
- prompt	0	1	2	3
- Log	0	1-2	3-4	5
Overall				30
- Does the program work	0	1-4	5-8	9-10
- Calculations and output correct	0	1-4	5-8	9-10
- Coding style	0	1-4	5-8	9-10

CHECKLIST

Once you have completed your assessment, use the checklist below to ensure that you have covered all elements.

	Task/ Requirement	YES	NO
1	Did you create a zip file and name it correctly?		
2	Did you include three html files in the zip?		
3	Did you include the css file in the zip?		
4	Did you include both js files in the zip?		